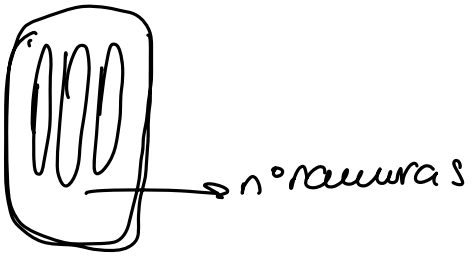
Ranura = [1, 2, 3...]



Ranura → LIBRE/ocupada → enum?
└→ nºPanTOST. → INT

TOSTADORA → RANURA → INT
└→ ESTADO → ENUM
└→ NIVEL → INT

```
public enum EstadoTostadora {
    ENCENDIDA
    APAGADA
}
```

//este enum hace que la tostadora tenga estado ENCENDIDA y APAGADA

```
Public class Tostadora {

    //atributos
    private int NivelC;
    private int NRanuras;
    private EstadoTostadora estado;


    //contructor para la tostadora
    Public Tostadora (int NivelC, int NRanuras, EstadoTostadora estado){

        this.NRanuras= NRANURAS;
        this.NivelC= NivelC;
        this.estado = estado.ENCENDIDA; --> Preguntar
        }
```

//metodos IniciarTostado, (void) FinalizarTostado, (void) IndicarNivel, (no void)

// contarRanurasLibres, (no void) RanuraMasPan, (no void)

```
Public void IniciarTostado (int nRanura) {
    //este metodo no devuelve nada
    if (this.estado == estado.ENCENDIDA) {
        this.ComienzaRanura();
```

```
else {                    → preguntar
excepcion }
```

```
Public void FinalizarTostado (Tostadora tostadora) {
```

```
Public int IndicarNivel
//este metodo retorna el nivel Calentamiento de la fosfadora

    return This.NivelC


Public int ContarRanuras Libres
//este metodo devuelve cuantas ranuras libres hay
// tiene que iterar en las ranuras y si
//  estado es libre suma uno al contador luego
/   return contador
```

Public Class Ranura

```
Public enum EstadoRanura {
        LIBRE
        OCUPADA
    }
```

```
// atributos
private int NPTost;
private EstadoRanura estador;

// constructor

// Public Ranura    // public o private?
Public Ranura ( int NPTost, EstadoRanura estador ) {

    this. NPTost = NPTost;
    this. estador = estador.LIBRE;

// metodos  Ranura con mas Pan (RMPan)

Public int RMPan ( ) {

    // TOSTADORA. NRanuras es una lista/array → busco el max?
    if (this. estador == estado-LIBRE) {
```